

Methodology

1. Overview of Proposed Methodology

This part introduces a two-stage deep learning model of malware detection that can simultaneously classify known malware families in closed sets and detect new malware variants in open sets. Phase I uses SimCLR self-supervised contrastive pre-training on malware visualization images; Phase II uses a prototype-based metric classifier that is fine-tuned. Confidence-thresholding mechanism allows open-set rejection without architectural adaptation. The entire end-to-end workflow is shown in Figure 1 and each of the stages will be discussed in the following subsections.

Detailed Workflow Diagram

Stage	Process	Key Operations / Components
1	Dataset Acquisition	Mallmg (Train/Val/Test) + Blended Malware Image Dataset (Open-Set Test)
2	Preprocessing & Augmentation	Grayscale → Resize 224×224 → Flip / Jitter → Tensor Normalization
3	Phase I — SimCLR Pre-Training	ResNet-18 Backbone NT-Xent Loss 20 Epochs Adam (lr=1e-3)
4	Feature Extraction	512-d Embedding Space Projection Head Discarded Post-Training
5	Phase II — Prototype Classifier	Learnable Prototypes (K=20) Cosine Similarity Weighted CE 30 Epochs
6	Threshold Calibration	Confidence Sweep $\delta \in [0.50, 0.95]$ Balanced Open-Set Score Criterion
7	Dual-Mode Evaluation	Closed-Set: Accuracy, F1, ROC-AUC Open-Set: URR, KAR, BOS

Figure 1. End-to-end workflow of the proposed malware detection framework.

2. Research Design

This study adopts an experimental deep learning research design that integrates self-supervised learning, metric learning, and open-set recognition within a unified and reproducible evaluation framework. The research follows a quantitative approach, where model performance is evaluated using standard classification and detection metrics.

The experimental pipeline is organized into four sequential phases:

I. Representation Learning : The SimCLR self-supervised contrastive learning framework is used to train a ResNet-18 backbone. It is also because the phase enables the model to acquire meaningful representation of features without having to work with massive labelled datasets.

II. Prototype-Based Classification: A prototype based classifier is then fine tuned on the encoder. Training is performed only on the samples that are known malware.

III. Open-Set Detection :The scores of the cosine similarity to the prototype classifier are applied to a confidence based thresholding system. This allows the model to identify and eliminate invisible malware samples as unknown.

IV. Evaluation: The model is also evaluated using a held-out test set using the close set measures of accuracy, F1-score, and ROC-AUC and the open set measures of unknown rejection rate and balanced open-set score.

The controlled evaluation and reproducibility are guaranteed by the experimental design. The individual components of the pipeline, like self-supervised pre-training, prototype learning and threshold selection are evaluated in a fixed experimental setting with a fixed random seed (seed =42)

3. Dataset Description

3.1 Data Sources

Three publicly available malware visualization datasets are used, all of which are developed on the technique described by Nataraj et al. [1], where raw binary executables are transformed into 2D grayscale images with consecutive bytes being interpreted as pixel intensities. Table 1 summarizes the datasets.

Dataset	Classes	Samples	Format	Source
Mallmg (9010 split)	25	~9,339 / ~1,035	PNG Grayscale	Kaggle / Figshare
Blended Malware Image Dataset	31	~5,000+	PNG Grayscale	Kaggle
Mallmg (updated val. set)	25	~924	PNG Grayscale	Figshare

Table 1. Summary of datasets used in the proposed malware detection framework.

3.2 Dataset Splits and Open-Set Configuration

All the classes are divided into 20 known (closed-set) classes that are used in training and validation, and other classes are used as unseen (open-set) classes to evaluate the detection performance. The training set is based on the 90 percent Mallmg partition (known only); the validation set is used in tuning thresholds to both known and unseen classes; the hold-out test set (924 samples) gives the final closed-set and open-set performance.

4. Data Preprocessing and Augmentation

All input images are processed through a standardized preprocessing pipeline. During training, the pipeline includes:

- conversion to single-channel grayscale format;
- resizing to 224×224 pixels using bilinear interpolation;
- application of random horizontal flipping with a probability of 0.5;
- color jittering with brightness, contrast, saturation, and hue parameters set to 0.1;
- conversion to normalized tensor values within the range $[0, 1]$.

In evaluation, the preprocessing pipeline does not include the process of horizontal flipping and color jittering as augmentation methods. This is to make sure that augmentation is simply a regularization procedure in training and does not affect the evaluation outcomes. In order to deal with the class imbalance issue with the Mallmg data, the inverse-frequency class weights are integrated into the cross-entropy loss. The weight of class c in class c .

5. Feature Extraction using ResNet-18

The encoder backbone is a modified ResNet-18 [4], selected for its favorable balance between representational capacity and computational efficiency on 224×224 grayscale inputs. The standard first convolutional layer, designed for three-channel RGB input, is replaced with a single-channel equivalent:

$$\text{Conv1: } (in=1, out=64, kernel=7 \times 7, stride=2, padding=3) \quad (2)$$

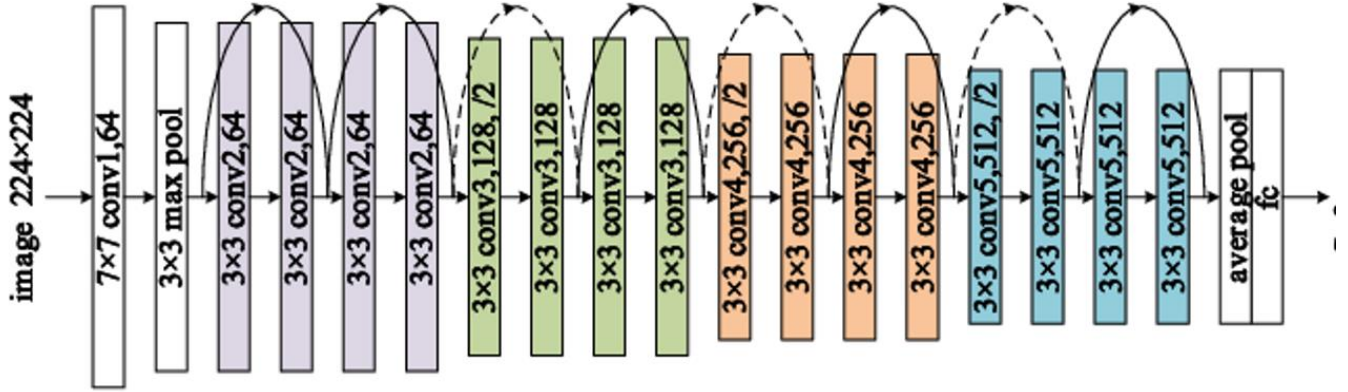


Figure 1. Encoder ResNet-18 architecture

The fully connected classification head is replaced with an identity mapping, yielding 512-dimensional feature embeddings shared by both the SimCLR projection head (Phase I) and the prototype classifier (Phase II). Table 2 summarizes the complete architecture.

Component	Output Dimension	Description
Input Layer	$1 \times 224 \times 224$	Single-channel grayscale malware image
Conv1 (Modified)	64 filters, 7×7 , stride 2	Adapted for single-channel input
Residual Blocks 1–4	$4 \times$ Residual Stacks	Hierarchical feature extraction
Global Avg. Pooling	512-d vector	Spatial feature aggregation
Encoder Output	512-d embedding	Shared by SimCLR and prototype heads
SimCLR Projection Head	$512 \rightarrow 256 \rightarrow 128$ (MLP)	NT-Xent contrastive projection (Phase I)
Prototype Layer	$K \times 512$ ($K = 20$)	Learnable class prototypes (Phase II)
Cosine Similarity Output	20-d similarity vector	Distance to each class prototype

Table 2. Encoder architecture and classification head configuration.

6. Phase I — Self-Supervised Pre-Training (SimCLR)

SimCLR [2] is adopted because it requires no memory banks or momentum encoders, simplifying the training pipeline, and achieves competitive performance with minimal hyperparameter tuning. For each training image x , two augmented views form a positive pair: $z_1 = \text{SimCLR}(x)$ and $z_2 = \text{SimCLR}(\text{flip}(x))$. The encoder output $h \in \mathbb{R}^{512}$ is projected through a two-layer MLP to produce the contrastive embedding $z \in \mathbb{R}^{128}$:

$$z = \text{normalize}(W_2 \cdot \text{ReLU}(W_1 \cdot h)), \quad W_1 \in \mathbb{R}^{256 \times 512}, \quad W_2 \in \mathbb{R}^{128 \times 256} \quad (3)$$

The NT-Xent loss maximizes agreement between positive pairs while pushing apart all other pairs in the batch:

$$\ell(i, j) = -\log [\exp(\text{sim}(z_i, z_j)/\tau) / \sum_{k \neq i} \exp(\text{sim}(z_i, z_k)/\tau)] \quad (4)$$

$$L_{\text{NT-Xent}} = (1/2N) \sum_{i=1}^{2k} \ell(i, j(i)) \quad (5)$$

where $\text{sim}(u, v) = u^T v / (\|u\| \cdot \|v\|)$ is cosine similarity and $\tau = 0.5$ is the temperature hyperparameter.

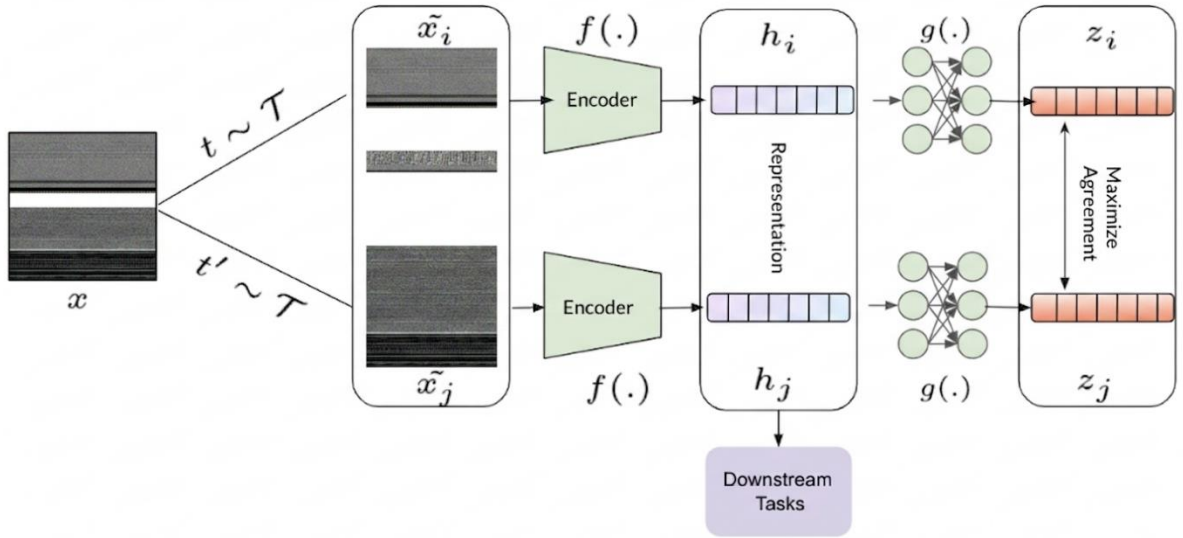


Figure 2. SimCLR Framework Architecture

Algorithm 1 details the full training procedure.

Algorithm 1: SimCLR Self-Supervised Pre-Training

Input: Training set D , Encoder $f(\cdot)$, Projector $g(\cdot)$

Output: Pre-trained encoder weights θ_f

Init: $f(\cdot) = \text{ResNet-18 (1-channel)}$; $g(\cdot) = \text{MLP}(512 \rightarrow 256 \rightarrow 128)$
Adam(lr=1e-3); ReduceLROnPlateau(patience=2, factor=0.5)

```

FOR epoch = 1 TO 20 DO:
  FOR each mini-batch  $\{x_i\}$  from D DO:
     $z_1 \leftarrow \text{normalize}(g(f(x)))$ ;  $z_2 \leftarrow$ 
     $\text{normalize}(g(f(\text{flip}(x))))$ 
     $L \leftarrow \text{NT-Xent}(z_1, z_2, \tau=0.5)$ 
    Update  $\theta_f, \theta_g$  via backpropagation
  END FOR
  Update LR scheduler with epoch loss
END FOR;

Return:  $\theta_f$ 

```

Algorithm 1. SimCLR self-supervised pre-training procedure.

7. Phase II — Prototype Classifier Training

Following Phase I, the encoder is fine-tuned with a prototypical network [3]. One learnable prototype vector $p_k \in \mathbb{R}^{512}$ per known class represents the idealized embedding-space centroid for that class. Classification scores are computed via cosine similarity:

$$s_k(x) = f(x)^T p_k / (\|f(x)\| \cdot \|p_k\|) \quad (6)$$

The classifier is optimized using class-weighted cross-entropy loss (weights from Eq. 1):

$$L_{\text{proto}} = - \sum_k w_k \cdot y_k \cdot \log(\text{softmax}(s(x))_k) \quad (7)$$

Cosine similarity is preferred over Euclidean distance as it is scale-invariant and bounded in $[-1, 1]$, providing a directly interpretable confidence score for open-set thresholding. Standard softmax classifiers produce overconfident predictions for out-of-distribution inputs; prototype metric learning yields lower, more reliable confidence for inputs distant from all known class prototypes. Algorithm 2 details the procedure.

Algorithm 2: Prototype Classifier Training

```

Input: Training set D_train, Pre-trained encoder  $f(\cdot)$ ,
Prototypes P

Output: Fine-tuned  $\theta_f$  and P

Init: P  $\sim N(0,1)$ ;  $\theta_f$  from Phase I
      Adam(lr=1e-4, wd=1e-5); ReduceLROnPlateau(patience=3,
factor=0.5)

FOR epoch = 1 TO 30 DO:
  FOR each (x, y) in D_train DO:
    h  $\leftarrow f(x)$  [512-d encoder
embedding]
     $s_k \leftarrow \text{cosine\_sim}(h, p_k), \quad k=1..K$ 
    L  $\leftarrow \text{weighted\_cross\_entropy}(s, y, \text{class\_weights})$ 
    Update  $\theta_f$  and P via backpropagation
  END FOR

```

```

    Validate on D_val; update LR scheduler
END FOR;

Return:  $\theta_f$ , P

```

Algorithm 2. Prototype classifier fine-tuning procedure.

8. Open-Set Detection and Threshold Calibration

Open-set detection is obtained by thresholding the highest score in the maximum cosine similarity with all the class prototypes. The decision rule is:

$$decision(x) = \underset{k}{\operatorname{argmax}} s_k(x) \text{ if } \max_k s_k(x) \geq \delta; \text{ else } UNKNOWN \dots \dots (8)$$

Known malware inputs are similar to their correct prototype with high cosine similarity, and novel inputs are similar to all prototypes with uniformly low similarity below 0.05. The best threshold δ is obtained through a validation sweep over δ 0.50, 0.55, etc. 0.95 by maximizing the Balanced Open-Set Score:

$$BOS(\delta) = [KnownAcc_with_rejection(\delta) + URR(\delta)] / 2 \dots \dots (9)$$

In which KnownAcc with rejection = the proportion of known samples that are accepted ($\text{conf} \geq 0.05$) and correctly classified, and URR = Unknown Rejection Rate (proportion of unknown samples rejected correctly). This formulation eliminates the over-rejection and over-acceptance bias.

9. Evaluation Strategy

The evaluation framework covers both closed-set and open-set metrics. Closed-set evaluation applies argmax over cosine similarity scores (no thresholding) on the 20 known test classes. Open-set evaluation applies δ^* to both known and unknown test samples. Table 3 summarizes all metrics employed.

Metric	Definition	Purpose
Accuracy	Correct / Total predictions	Overall classification quality
Macro F1-Score	Unweighted mean of per-class F1	Performance under class imbalance
ROC-AUC (per class)	Area under TPR–FPR curve	Threshold-free ranking quality
Confusion Matrix	$N \times N$ prediction grid	Detailed per-class error analysis
Unknown Rejection Rate (URR)	Rejected unknowns / Total unknowns	Open-set detection power
Known Accept Rate (KAR)	Accepted knowns / Total knowns	Closed-set gate reliability
Balanced Open-Set Score (BOS)	$(KnownAcc_rejection + URR) / 2$	Unified open/closed-set criterion

Table 3. Evaluation metrics for closed-set classification and open-set detection.

10. Training Configuration Summary

Hyperparameter	Value	Rationale
Batch Size	64	Optimal memory–accuracy trade-off
SSL Pre-Training Epochs	20	Sufficient for NT-Xent convergence
Prototype Training Epochs	30	Stable prototype centroid convergence
SimCLR Temperature (τ)	0.5	Standard SimCLR; sharpens similarity distribution
SSL Learning Rate	1×10^{-3} (Adam)	Aggressive initial representation learning
Prototype Learning Rate	1×10^{-4} (Adam)	Fine-grained prototype fine-tuning
Weight Decay	1×10^{-5}	Regularization against prototype overfitting
LR Scheduler	ReduceLROnPlateau (patience=3)	Adaptive decay on validation loss plateau
Input Resolution	224×224 pixels	Standard ResNet-18 input size
Embedding Dimension	512-d	ResNet-18 penultimate representation
Projection Head Dim.	128-d	SimCLR standard projection output
Known Classes (K)	20	Closed-set class count
Random Seed	42	Full reproducibility

Table 4. Complete hyperparameter and training configuration.

11. Methodological Justification

The suggested framework has three major limitations to the current literature on malware detection. First, the issue of label scarcity: the majority of previous systems need large and completely labeled datasets. SimCLR pre-training with unlabeled malware images allows label-efficient representation learning, and it is less reliant on annotation. Second, on the issue of open-set vulnerability: most previous detectors assume the closed-set and do not give an error when new families are introduced after deployment. This work explicitly models and rejects unknown inputs by the confidence-thresholding mechanism. Third, on the issue of class imbalance: inverse-frequency class weighting guarantees sufficient gradient signal to minority malware families, and avoids model collapse to majority-class decisions.

It is better than MoCo, BYOL, and DINO because it does not use memory banks or momentum encoders and horizontal flip is a natural positive-pair strategy that is compatible with malware byte-pattern asymmetry. ResNet-18 is selected because of its residual connections that alleviate vanishing gradients and its 512-d penultimate representation, which is still amenable to prototype metric learning. Cosine similarity is more desirable than Euclidean distance because it is scale-invariant and is in $[-1, 1]$, which allows uniform thresholding of inputs of different magnitude.